Recursividade

Prof. Ricardo Mesquita

Conceito

- Fundamental em Matemática e Ciência da Computação
  - Uma função *recorrente* é definida em termos dela mesma
  - Um programa *recursivo* é um programa que chama a si mesmo

***Note**: usamos o termo "recorrência" em matemática e "recursividade" em computação.*

- Exemplos:

  Números naturais, Função fatorial, Árvore…

- **Conceito poderoso**: define conjuntos infinitos com *comandos* finitos!

Recursividade

- *Definição*: dentro do corpo de uma função, chamar novamente a própria função
  - *recursividade direta*: a função A chama a própria função A
  - *recursividade indireta*: a função A chama uma função B que, por sua vez, chama a função A

Recursividade

- A recorrência é uma estratégia que pode ser utilizada sempre que o cálculo de uma função para o valor n, pode ser descrita a partir do cálculo desta mesma função para o termo anterior $(n - 1)$.

Exemplo – Função fatorial:

$n! = n * (n-1) * (n-2) * (n-3) *....* 1$

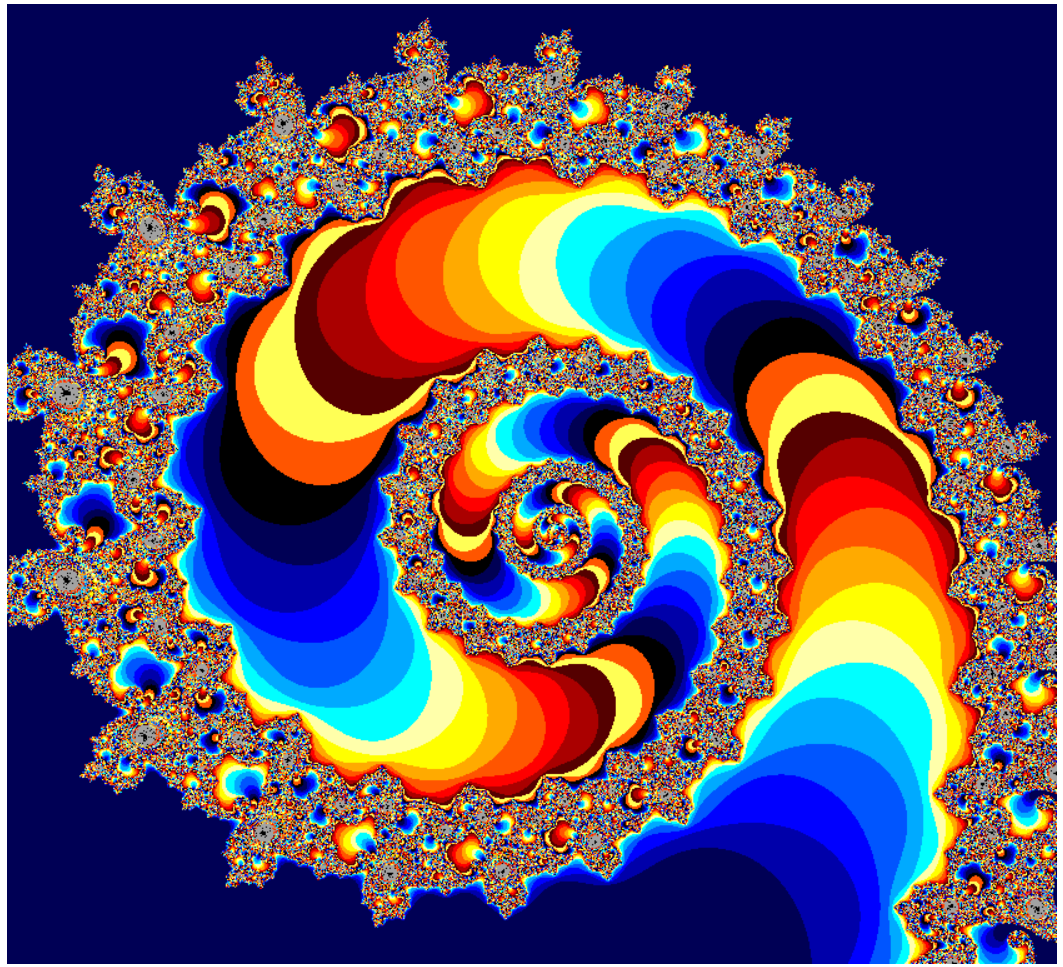
$(n-1)! = (n-1) * (n-2) * (n-3) *....* 1$

logo:

$n! = n * (n-1)!$

Condição de Parada

- Nenhum programa nem função pode ser exclusivamente definido por si mesmo:
  - O programa entraria em um *loop infinito*.
  - A função teria uma definição circular, infinita.
- ***Condição de parada***:
  - Permite que o procedimento pare de ser executado em algum momento
  - Exemplo: $F(x) > 0$ onde x é decrescente
- Objetivo: estudar recursividade como ferramenta *prática*!



Equação de Recorrência

- Define-se uma função de complexidade $f(n)$ desconhecida
  - n mede o tamanho dos argumentos para o procedimento em questão
- Identifica-se a **equação de recorrência** $T(n)$:
  - Especifica-se $T(n)$ como uma função dos termos anteriores
  - Especifica-se a condição de parada (e.g., $T(1)$)

Equação de Recorrência

- Equação de recorrência para a função fatorial:

$$T(0) = \begin{cases} 1 \\ n * T(n-1) \end{cases}$$

Condição de parada!

$$T(n) = n * T(n-1)$$
$$T(n-1) = (n-1) * T(n-2)$$
$$T(n-2) = (n-2) * T(n-3)$$
$$\ldots$$
$$T(1) = 1 * T(0)$$

Execução Computacional

Para ***cada*** chamada de uma função, recursiva ou não, os parâmetros e as variáveis locais são empilhados na pilha de execução.

Internamente, quando qualquer chamada de função é feita dentro de um programa, é criado um **registro de ativação** na **pilha de execução** do programa.

O registro de ativação armazena os parâmetros e variáveis locais da função bem como o "ponto de retorno" no programa ou subprograma que chamou essa função.

Ao final da execução dessa função, o registro é desempilhado e a execução volta ao subprograma que chamou a função.

Exemplo: Cálculo de 4!

$$\begin{cases} T(0) = 1 \\ T(n) = n * T(n-1) \end{cases}$$

Implementação:

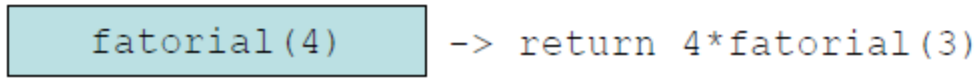
```
int fatorial(int n)
{
  if (n == 0)
    return 1;
  else
    return n * fatorial(n-1);
}
```

Exemplo: Cálculo de 4!

```
int fatorial(int n)
{
  if (n == 0)
    return 1;
  else
    return n * fatorial(n-1);
}
```

Pilha de Execução

Empilha `fatorial(4)`



Exemplo: Cálculo de 4!

```
int fatorial(int n)
{
  if (n == 0)
    return 1;
  else
    return n * fatorial(n-1);
}
```

Pilha de Execução

Empilha `fatorial(3)`

`fatorial(3)`	`-> return 3*fatorial(2)`
`fatorial(4)`	`-> return 4*fatorial(3)`

Exemplo: Cálculo de 4!

```
int fatorial(int n)
{
  if (n == 0)
    return 1;
  else
    return n * fatorial(n-1);
}
```

Pilha de Execução

Empilha `fatorial(2)`

`fatorial(2)`	`-> return 2*fatorial(1)`
`fatorial(3)`	`-> return 3*fatorial(2)`
`fatorial(4)`	`-> return 4*fatorial(3)`

Exemplo: Cálculo de 4!

```
int fatorial(int n)
{
  if (n == 0)
    return 1;
  else
    return n * fatorial(n-1);
}
```

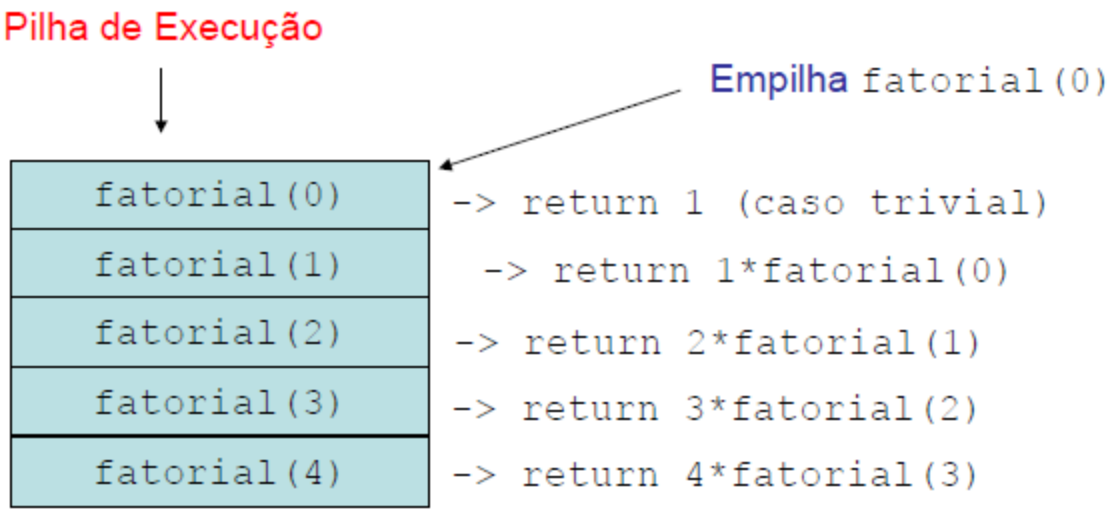
Pilha de Execução

Empilha `fatorial(1)`

`fatorial(1)`	`-> return 1*fatorial(0)`
`fatorial(2)`	`-> return 2*fatorial(1)`
`fatorial(3)`	`-> return 3*fatorial(2)`
`fatorial(4)`	`-> return 4*fatorial(3)`

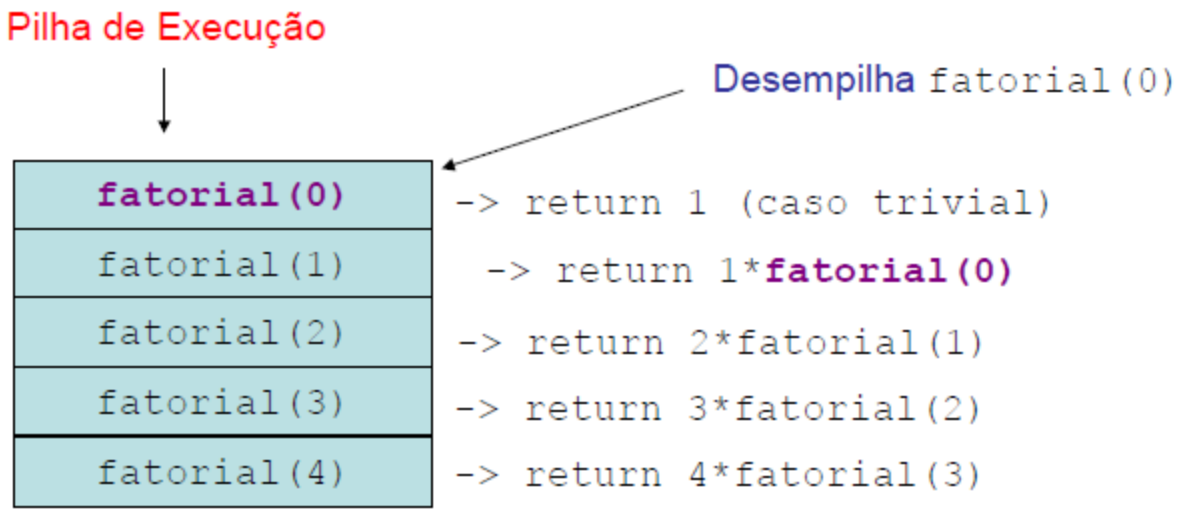
Exemplo: Cálculo de 4!

```
int fatorial(int n)
{
  if (n == 0)
    return 1;
  else
    return n * fatorial(n-1);
}
```



Exemplo: Cálculo de 4!

```
int fatorial(int n)
{
  if (n == 0)
    return 1;
  else
    return n * fatorial(n-1);
}
```



Exemplo: Cálculo de 4!

```
int fatorial(int n)
{
  if (n == 0)
    return 1;
  else
    return n * fatorial(n-1);
}
```

Pilha de Execução

Desempilha `fatorial(1)`

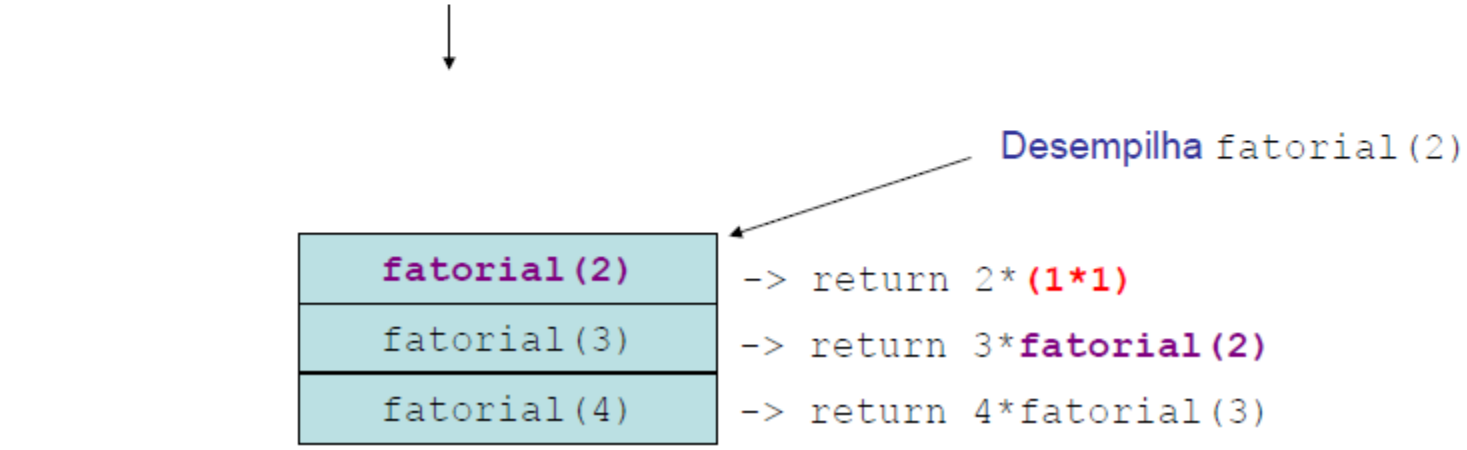
fatorial(1)	-> return 1*1
fatorial(2)	-> return 2*fatorial(1)
fatorial(3)	-> return 3*fatorial(2)
fatorial(4)	-> return 4*fatorial(3)

Exemplo: Cálculo de 4!

- Considere, novamente, o exemplo para 4!:

```
int fatorial(int n)
{
  if (n == 0)
    return 1;
  else
    return n * fatorial(n-1);
}
```

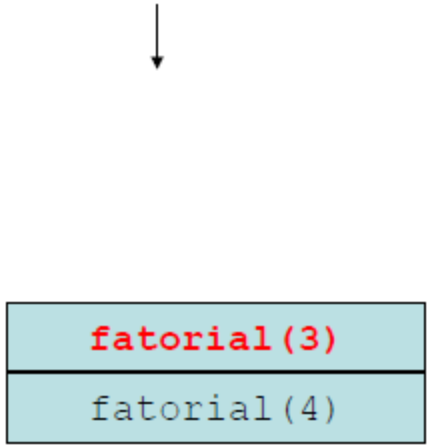
Pilha de Execução



Exemplo: Cálculo de 4!

```
int fatorial(int n)
{
  if (n == 0)
    return 1;
  else
    return n * fatorial(n-1);
}
```

Pilha de Execução



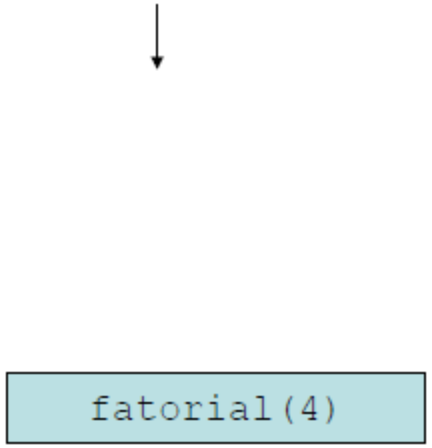
Desempilha `fatorial(3)`

```
-> return 3*(2*1*1)
-> return 4*fatorial(3)
```

Exemplo: Cálculo de 4!

```
int fatorial(int n)
{
  if (n == 0)
    return 1;
  else
    return n * fatorial(n-1);
}
```

Pilha de Execução



Exemplo: Cálculo de 4!

```
int fatorial(int n)
{
  if (n == 0)
    return 1;
  else
    return n * fatorial(n-1);
}
```

Resultado = 24

Observações

- A recursividade nem sempre é a melhor solução, mesmo quando a definição matemática do problema é feita em termos recursivos!
  - **Motivo**: grande complexidade (sobretudo espacial)
- Deve-se usar recursividade ***com sabedoria...***
- Algumas estruturas de dados são inerentemente recursivas, portanto, nesses casos, a recursividade é a primeira solução
- *Sempre que possível, prefira soluções não-recursivas!... Em geral, são mais rápidas!*

Exercícios

1. **Torres de Hanói**

O problema das Torres de Hanói é um quebra-cabeça matemático que consiste em uma base com três pinos e um número de discos de diferentes tamanhos.

O objetivo é transferir todos os discos do pino de origem para o pino de destino, usando um terceiro pino como auxiliar, seguindo três regras fundamentais:

- Um por vez: Apenas um disco pode ser movido em cada jogada.
- Topo da pilha: Cada movimento consiste em retirar o disco superior de uma das pilhas e depositá-lo no topo de outra pilha.
- Ordem de tamanho: Um disco maior nunca pode ser colocado sobre um disco menor.

Apresente uma solução computacional para este problema.

Exercícios

2. **O Problema de Josephus**

Imagine um grupo de n pessoas dispostas em um círculo. A partir de uma pessoa específica, a contagem começa em uma direção fixa. A cada passo, pula-se um determinado número k de pessoas e a k-ésima pessoa é eliminada do círculo. O processo continua, fechando o círculo com os sobreviventes, até que reste apenas uma pessoa. O desafio é determinar a posição inicial em que você deve estar para ser o último sobrevivente.

Apresente uma solução computacional para este problema.

Praticar!